

Understanding daphne-firmware

Architecture, build flow, verification strategy, and trigger studies

Manuel Arroyave

April 2026

What this repo is

One-sentence version

- ▶ `daphne-firmware` is the repository that turns the historical DAPHNE mezzanine Vivado design into something that can be built, reasoned about, verified, and eventually refactored without losing the working hardware path.

Why it exists

- ▶ The starting point was a working but monolithic firmware tree.
- ▶ The immediate goal was **not** a rewrite. The goal was to preserve the shipped path while making the design understandable and testable in pieces.
- ▶ The repo therefore contains both:
 - ▶ the imported compatibility lane under `ip_repo/` and `xilinx/`
 - ▶ the newer repo-owned modular, FuseSoC, and formal layers

Where we started

- ▶ The original firmware lived as a large Vivado-centered codebase with implicit source lists, generated IP dependencies, and subsystem boundaries that were mostly tribal knowledge.
- ▶ That structure is normal for a lab firmware project, but it makes review, reproducibility, regression isolation, and verification unnecessarily difficult.
- ▶ The project started from one hard constraint:
 - ▶ **do not break the working K26C path while making the design cleaner.**
- ▶ In practice, this means the migration strategy is additive:
 - ▶ preserve the qualified implementation flow,
 - ▶ expose explicit boundaries,
 - ▶ add local tests and proofs,
 - ▶ replace internals only after the boundary is explicit.

Purpose of the development

Engineering objective

- ▶ Convert a monolithic firmware image into a maintainable hardware/software artifact with a stable build contract.

Concrete sub-goals

- ▶ keep the K26C hardware image buildable and deployable;
- ▶ make board, feature, and test intent explicit;
- ▶ isolate analog, frontend, timing, trigger, and transport ownership;
- ▶ make leaf-level control behavior proof-friendly;
- ▶ document the build from `git clone` to `.bit/.bin/.xsa/.dtbo`.

Working rule

ship path alive $\wedge$ boundary explicit $\wedge$ verification added first

This is why the repo has both a compatibility layer and a modular layer.

April 20, 2026

What we achieved

- ▶ Imported the existing firmware tree into a versioned repo with explicit board and platform metadata.
- ▶ Added FuseSoC cores for platform, feature, smoke-test, and formal targets.
- ▶ Added repo-owned subsystem wrappers under rtl/isolated/.
- ▶ Added formal harnesses for control and boundary contracts.
- ▶ Added repo-owned DT overlay packaging and deployment artifacts.
- ▶ Documented both a native Linux build path and a WSL + Windows toolchain path.
- ▶ Validated the operational baseline on hardware:
 - ▶ routed-clean baseline: a389fcd
 - ▶ current main: 64d38d1
 - ▶ overlay load, clock bring-up, daphne-server, oscilloscope mode
- ▶ This repo is now more than documentation and scaffolding: it owns a real build-to-overlay workflow.

Why FuseSoC

Problem

- ▶ A pure Vivado project is a poor long-term source-of-truth for a modular firmware project.

FuseSoC gives us

- ▶ explicit source graphs;
- ▶ board/platform manifests;
- ▶ target selection for smoke, formal, and implementation;
- ▶ less dependence on hand-maintained file lists;
- ▶ a place to express intent above vendor Tcl.

The point is not ideology

- ▶ Vivado still performs implementation.
- ▶ FuseSoC owns the *description of what we mean to build*.

Why formal verification

What formal is used for here

- ▶ not for proving the entire imported design,
- ▶ but for proving small, stable contracts at boundaries and AXI-lite leaves.

Why this matters

- ▶ During migration, most bugs are not “DSP math is wrong” bugs.
- ▶ They are reset, gating, readback, enable, and illegal-write bugs.
- ▶ Those are exactly the cases formal handles well.

Examples already covered

- ▶ threshold register semantics,
- ▶ frontend and trigger readiness gates,
- ▶ analog-control, timing, and Hermes boundary contracts,
- ▶ delay-line replacement contracts.

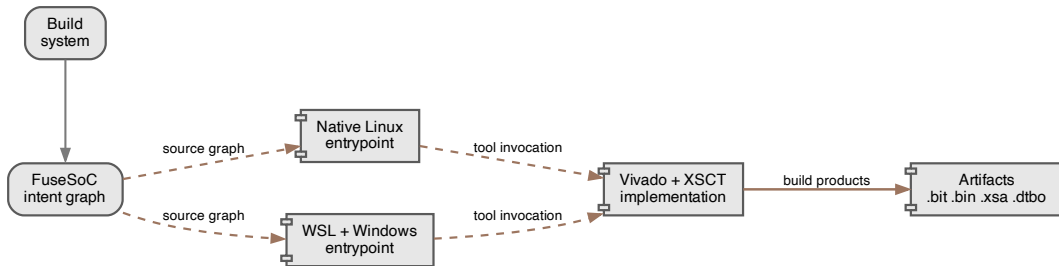
Module map

Module	Responsibility
Control plane	PS-visible register map, AXI-lite leaves, stable software contract
Analog/frontend control	AFE and DAC configuration, configuration-ready semantics
Frontend boundary	LVDS deserialization, IDELAY/bitflip, trusted alignment promotion
Timing subsystem	endpoint timing ingress, clock selection, lock/ready/timestamp validity
Trigger pipeline	filters, self-trigger logic, thresholds, peak descriptors, STC3 record build
Transport / Hermes	packet export and Ethernet-linked readout transport
Verification layer	smoke tests, formal contracts, boundary assertions, build checks

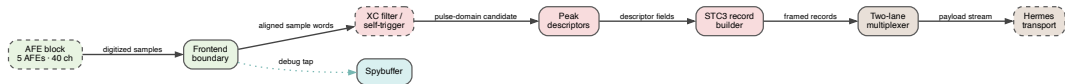
Repository organization

- ▶ Imported compatibility path: `ip_repo/`, `xilinx/`
- ▶ Modular repo-owned path: `rtl/isolated/`, `cores/`, `formal/`, `docs/`

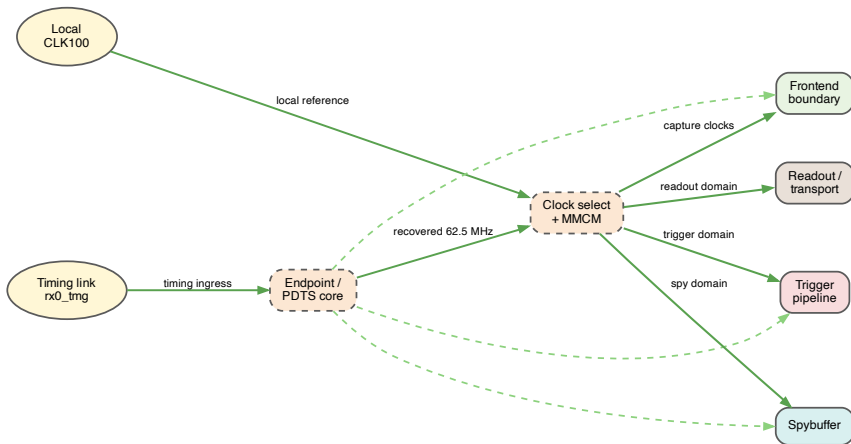
Build flow



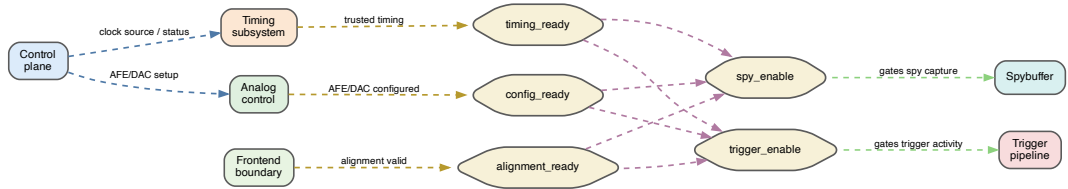
Runtime acquisition path



Clock and timing topology



Readiness contracts



Build on a pure Linux machine

Path rule

- ▶ clone to a short path such as $\tilde{w}/d$
- ▶ avoid spaces and deep nested directories

Clone + build

```
mkdir -p ~/w
git clone git@github.com:DUNE-DAQ/daphne-firmware.git ~/w/d
cd ~/w/d
export XILINX_SETTINGS_SH=/path/to/Vivado/2024.1/settings64.sh
export XILINX_VITIS_SETTINGS_SH=/path/to/Vitis/2024.1/settings64.sh
export DAPHNE_BOARD=k26c DAPHNE_ETH_MODE=create_ip
./scripts/remote/run_remote_vivado_chain.sh
```

Expected output

- ▶ xilinx/output-<gitsha>/ with .bit, .bin, .xsa, .dtbo, overlay zip, and timing reports

Build on Windows + WSL

Qualified arrangement

- ▶ shell in WSL2
- ▶ Vivado 2024.1 and Vitis 2024.1 installed on Windows
- ▶ short mirrored path: C:\w\d and /mnt/c/w/d

Clone in WSL

```
mkdir -p /mnt/c/w
git clone git@github.com:DUNE-DAQ/daphne-firmware.git /mnt/c/w/d
cd /mnt/c/w/d
export DAPHNE_BOARD=k26c DAPHNE_ETH_MODE=create_ip
./scripts/wsl/check_windows_xilinx.sh
./scripts/wsl/run_wsl_vivado_chain.sh
```

Why the short path matters

- ▶ Vivado, DTG, and generated IP trees create very deep paths.
- ▶ Short paths reduce avoidable tool failures.

Mathematical model: baseline and filter preprocessing

- ▶ Let $x[n]$ be the sampled AFE waveform.
- ▶ The trigger path first estimates a slow baseline and then forms a pulse-domain signal for triggering and descriptors.

Firmware low-pass baseline tracker

$$b[n] = b[n-1] + 2^{-k}(x[n] - b[n-1]), \quad k = 26$$

- ▶ This is the behavior of `k_low_pass_filter.v`.
- ▶ It is a very slow IIR baseline estimator.

Centered / high-pass view

$$d[n] \approx x[n] - b[n]$$

- ▶ Depending on configuration, the polarity may be inverted before the matched filter and descriptor stages.

Mathematical model: matched filter and self-trigger

Cross-correlation stage

$$y[n] = \sum_{i=0}^{31} h[i] d[n - i]$$

- ▶ $h[i]$ is the fixed 32-tap template in `st_xc.vhd`.
- ▶ The RTL uses a transposed FIR structure with pipeline registers matching the DSP chain.

Trigger condition

$$\text{trig}[n] = 1$$

when $y[n] > T$, $y[n - 1] > T$, and $y[n - 2] \leq T$.

- ▶ This makes the trigger a one-cycle pulse on a threshold crossing that persists for two consecutive samples.
- ▶ In the integrated path, this condition is then combined with optional ad-hoc triggers and a fixed latency delay.

Mathematical model: peak descriptors

Purpose

- Once a pulse is detected, the descriptor path compresses the waveform into metadata useful for higher-level filtering and transport.

Peak-detection logic

$$s[n] = x[n] - x[n - L], \quad L \in \{16, 20\}$$

$$\text{peak if } s[n] \leq \theta_s$$

- This is the slope-threshold logic in `PeakDetector_SelfTrigger.vhd`.
- An FSM inhibits immediate retriggering until the slope recovers.

Descriptor observables

$$A_{\max} = \max_t \{-a[t]\}, \quad t_{\max} = \arg \max_t \{-a[t]\}$$

$$Q = \sum_t \max(0, -a[t]), \quad W = \text{samples in detection window}, \quad N_{\text{pk}} = \text{count of detected peaks}$$

XC emulation: what it is and what it is not

- ▶ The repo `daphne_mezz_xc_sim` contains a C++ model of the xcorr self-trigger path.
- ▶ It mirrors:
 - ▶ the 32-tap transposed FIR,
 - ▶ the trigger condition,
 - ▶ the baseline LPF option,
 - ▶ the CFD gate,
 - ▶ the CIEMAT descriptor path.
- ▶ This is useful for:
 - ▶ rapid waveform studies,
 - ▶ threshold and latency scans,
 - ▶ testing trigger logic without full implementation.
- ▶ It intentionally does **not** model:
 - ▶ Ethernet transport,
 - ▶ full shared-lane arbitration.

RTL trigger studies: dead time as a system property

Question

- ▶ If every channel sees the same nominal offered trigger rate, what dead time per channel should we expect?

Two complementary approaches

1. Nominal stochastic model

- ▶ uses the measured RTL constants for builder busy time and shared-lane service;
- ▶ scans the accepted dead fraction versus offered rate.

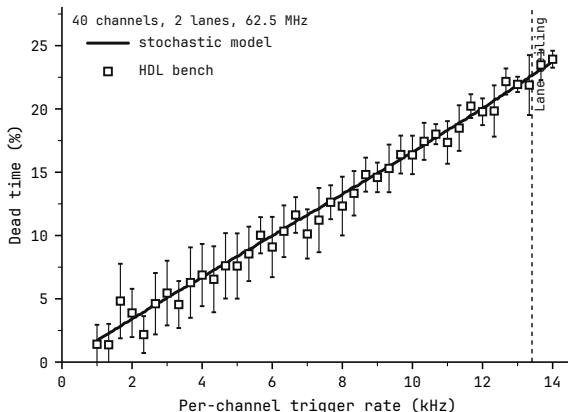
2. HDL multichannel bench

- ▶ instantiates the real record builder and mux logic;
- ▶ drives 40 channels with independent arrivals;
- ▶ measures accepted, dropped, and sent records.

Nature of the test

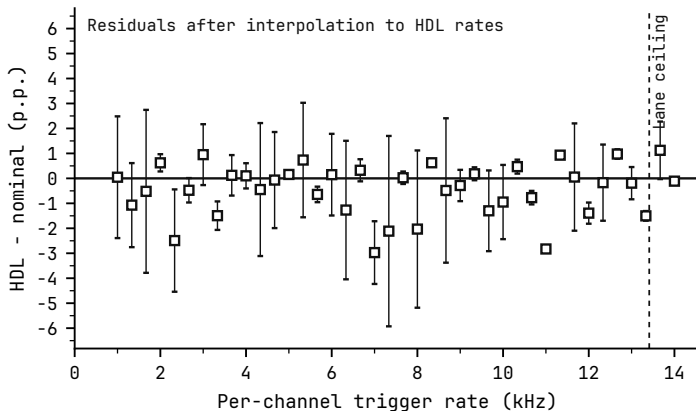
- ▶ This is a *steady-state queueing study* of the trigger and readout chain.

Trigger-study result: model versus RTL



Nominal model against the HDL 40-point, 8-repeat multichannel bench. HDL markers show the per-rate mean dead fraction, with sample standard deviation across repeats as the error bar. The same bench summary also stores mean/std for accepted, sent, busy-drop, and full-drop totals at each rate.

Residual view: where the model departs from RTL



Residuals are plotted as $\Delta(r) = \bar{f}_{\text{HDL}}(r) - f_{\text{nominal}}(r)$ after interpolation of the nominal curve onto the HDL rate points. The y-error bars reuse the HDL pointwise sample standard deviation; for the current scan, $\langle |\Delta| \rangle \approx 0.83$ p.p. and $\max |\Delta| \approx 2.97$ p.p.

Scope and limits

Operational scope

- ▶ qualified board path: K26C
- ▶ operational routed-clean baseline: a389fcd
- ▶ current repo tip: main at 64d38d1

Signal-model assumptions

- ▶ baseline subtraction is slow relative to pulse width;
- ▶ trigger decisions operate on a filtered pulse domain.

Dead-time-study assumptions

- ▶ homogeneous offered rate across channels;
- ▶ independent arrivals in the baseline study;
- ▶ steady-state measurement after warm-up.

What to remember

- ▶ `daphne-firmware` is a migration repo with one central rule: preserve the working hardware path while making the design understandable and verifiable.
- ▶ FuseSoC gives the source and target graph; Vivado still performs implementation.
- ▶ Formal is used where it is cost-effective: contracts, gates, and AXI-lite/control seams.
- ▶ The trigger path is algorithmic, not ad hoc:
 - ▶ slow baseline estimate,
 - ▶ matched-filter trigger,
 - ▶ descriptor extraction,
 - ▶ framed transport through shared readout lanes.
- ▶ The repo now contains both:
 - ▶ a hardware-proven build/deploy baseline,
 - ▶ and the analytical + HDL tools needed to study trigger behavior at RTL level.